

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Văn A

TIÊU ĐỀ KHOÁ LUẬN TỐT NGHIỆP

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN VĂN A

TIÊU ĐỀ KHOÁ LUẬN TỐT NGHIỆP

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Nguyễn Văn B

Cán bộ đồng hướng dẫn: TS. Nguyễn Văn C

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyen Van A

THESIS TITLE

BACHELOR'S THESIS

Major: Information Technology

Supervisor: Dr. Nguyen Van B

Co-Supervisor: Dr. Nguyen Van C

HANOI - 2026

Lời cảm ơn

Lời cam đoan

Tôi là ..., sinh viên lớp. Tôi xin cam đoan ...

Hà Nội, ngày tháng năm

Sinh viên

Nguyễn Văn A

Tóm tắt

Tóm tắt nội dung KL tốt nghiệp: Bài toán, phương pháp giải quyết, kết quả đạt được

Từ khóa: Liệt kê 3-5 từ khoá liên quan

Abstract

Summary of the graduation thesis: The problem statement, the proposed approach, and the achieved results.

Keywords: Liệt kê 3-5 từ khoá liên quan

Mục lục

Danh sách hình vẽ

Danh sách bảng

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
BiLSTM	Bidirectional Long Short Term Memory	Mô hình bộ nhớ dài hạn ngắn hạn hai chiều
BO	Buffer Overflow	Lỗi tràn bộ đệm

Chương 1

Đặt vấn đề

1.1 Bối cảnh và lý do chọn đề tài

Trong kỷ nguyên công nghệ số và sự phát triển mạnh mẽ của các thiết bị di động thông minh, thói quen tiếp cận thông tin và giải trí của con người đã trải qua những thay đổi sâu sắc. Một trong số đó là sự chuyển dịch từ việc đọc sách, truyện tranh in ấn truyền thống sang các nền tảng kỹ thuật số. Đặc biệt, đối với thể loại truyện tranh (manga, manhwa, comic), sự phát triển của Internet đã kéo theo sự ra đời của hàng loạt các dịch vụ lưu trữ và phân phối nội dung trực tuyến. Người dùng ngày càng có nhu cầu tiếp cận với hàng ngàn bộ truyện từ khắp nơi trên thế giới một cách nhanh chóng, tiện lợi mọi lúc mọi nơi thông qua chiếc điện thoại di động của mình.

Tuy nhiên, thực tế trải nghiệm người dùng trên các nền tảng đọc truyện hiện nay vẫn tồn tại nhiều hạn chế. Việc truy cập qua trình duyệt web trên thiết bị di động thường gặp phải tình trạng tối ưu hóa giao diện kém, tốc độ tải trang chậm do tải quá nhiều thành phần không cần thiết (quảng cáo, script theo dõi), và thiếu khả năng lưu trữ ngoại tuyến (offline) một cách hiệu quả. Hơn nữa, sự phân mảnh của các nguồn truyện khiến người dùng gặp khó khăn trong việc quản lý tập trung thư viện cá nhân, theo dõi tiến độ đọc, và nhận thông báo khi có chương mới.

Nhận thấy những vấn đề thiết thực trên, việc phát triển một ứng dụng di động chuyên biệt dành cho nền tảng Android, tập trung vào trải nghiệm đọc truyện mượt mà, quản lý thư viện thông minh và tích hợp các công nghệ hiện đại là một hướng đi vô cùng cấp thiết và mang nhiều ý nghĩa thực tiễn. Đề tài "Xây dựng ứng dụng đọc truyện tranh trực tuyến trên nền tảng Android" (MyBooksLibrary) được lựa chọn nhằm cung cấp một giải pháp toàn diện, khắc phục các nhược điểm của việc đọc truyện trên trình duyệt, đồng thời ứng dụng các kiến trúc phần mềm tiên tiến như Clean Architecture, MVVM cùng hệ sinh thái Jetpack Compose để đảm bảo hiệu suất và khả năng mở rộng của phần mềm.

1.2 Mô tả bài toán

Bài toán đặt ra cho đề tài là thiết kế và phát triển một ứng dụng đọc truyện tranh trên nền tảng hệ điều hành Android, có khả năng kết nối với một nguồn cung cấp dữ liệu truyện tranh trực tuyến (cụ thể là MangaDex API) để truy xuất danh sách truyện, thông tin chi tiết và dữ liệu hình ảnh của các chương truyện.

- **Đầu vào của bài toán:** Dữ liệu JSON được cung cấp từ các RESTful API của hệ thống MangaDex, bao gồm thông tin metadata của truyện (tên, tác giả, thể loại, mô tả), danh sách các chương, và đường dẫn URL tới các trang ảnh truyện. Ngoài ra, đầu vào còn bao gồm các tương tác của người dùng trên giao diện ứng dụng (tìm kiếm, thêm vào thư viện, vuốt trang, đánh dấu chương đã đọc).
- **Đầu ra của bài toán:** Một ứng dụng Android hoàn chỉnh với giao diện trực quan, hiển thị nội dung truyện tranh sắc nét, tối ưu hoá cho nhiều kích thước màn hình khác nhau. Hệ thống phải cho phép người dùng đọc truyện trực tiếp, quản lý danh sách truyện yêu thích, thống kê lịch sử đọc, và hỗ trợ tải dữ liệu ảnh về thiết bị để xem ngoại tuyến.
- **Các giả định:** Thiết bị di động của người dùng chạy hệ điều hành Android có phiên bản SDK tối thiểu là 30. Để thực hiện các thao tác lấy dữ liệu mới và đồng bộ hoá đám mây, thiết bị cần được kết nối với Internet. Đối với các chương truyện đã tải xuống, ứng dụng hoàn toàn có thể hoạt động ngoại tuyến.

1.3 Mục tiêu và phạm vi nghiên cứu

1.3.1 Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề tài là xây dựng thành công ứng dụng MyBooksLibrary với giao diện hiện đại, dễ sử dụng, đáp ứng tối đa nhu cầu đọc truyện tranh trên thiết bị di động. Cụ thể, các mục tiêu kỹ thuật bao gồm:

- Nghiên cứu và áp dụng thành thạo ngôn ngữ Kotlin, kết hợp với bộ công cụ xây dựng giao diện Jetpack Compose.

- Áp dụng kiến trúc Clean Architecture và mô hình MVVM (Model-View-ViewModel) nhằm tạo ra một mã nguồn mạch lạc, dễ bảo trì và có khả năng mở rộng cao.
- Sử dụng Dependency Injection (thông qua thư viện Hilt) để giảm sự phụ thuộc giữa các thành phần trong hệ thống.
- Tích hợp các thư viện xử lý bất đồng bộ (Coroutines/Flow) và giao tiếp mạng (Retrofit, OkHttp) một cách hiệu quả để đảm bảo hiệu năng, tránh tình trạng giật lag trên giao diện người dùng.
- Phát triển cơ chế quản lý dữ liệu cục bộ bằng Room Database để quản lý thư viện cá nhân và lịch sử đọc, đồng thời tích hợp Firebase để hỗ trợ xác thực người dùng và đồng bộ hoá dữ liệu đám mây.

1.3.2 Phạm vi nghiên cứu

- **Về nền tảng:** Đề tài chỉ giới hạn phát triển cho hệ điều hành di động Android.
- **Về dữ liệu:** Ứng dụng hiện tại tích hợp và lấy dữ liệu trực tiếp từ nguồn mở MangaDex API. Đề tài không đi sâu vào việc tự xây dựng backend server chứa nội dung truyện mà chỉ đóng vai trò là một client tiêu thụ API.
- **Về tính năng:** Ứng dụng tập trung vào các tính năng cốt lõi của một phần mềm đọc truyện: Khám phá truyện, tìm kiếm, hiển thị chi tiết, giao diện đọc (reader) linh hoạt, quản lý thư viện lưu trữ (thêm, xóa, đánh dấu tiến trình), tải xuống offline và đồng bộ hóa cơ bản.

1.4 Cấu trúc báo cáo

Báo cáo được bố cục thành 4 chương chính nhằm trình bày chi tiết và logic quá trình nghiên cứu, phát triển ứng dụng:

- **Chương 1: Đặt vấn đề.** Giới thiệu tổng quan về bối cảnh, lý do chọn đề tài, mô tả bài toán cần giải quyết, cùng với mục tiêu và phạm vi nghiên cứu.
- **Chương 2: Các công nghệ và thư viện sử dụng.** Tổng hợp các kiến thức cơ sở về ngôn ngữ lập trình, kiến trúc hệ thống (Clean Architecture, MVVM),

và phân tích vai trò của các thư viện cốt lõi (Jetpack Compose, Room, Retrofit, Hilt, Coil) được áp dụng trong dự án.

- **Chương 3: Phân tích yêu cầu và thiết kế hệ thống.** Liệt kê các tính năng của phần mềm, xác định các tác nhân, thiết kế sơ đồ Use Case và xây dựng sơ đồ tuần tự cho các luồng hoạt động chính yếu.
- **Chương 4: Đánh giá và kiểm thử.** Đưa ra phương pháp kiểm thử (Unit test), quá trình đánh giá tính ổn định, hiệu năng của ứng dụng và tổng kết các kết quả đạt được.

Chương 2

Các công nghệ và thư viện sử dụng

Chương này trình bày chi tiết về các nền tảng, ngôn ngữ lập trình, kiến trúc hệ thống và các thư viện cốt lõi được áp dụng trong quá trình xây dựng ứng dụng MyBooksLibrary. Đối với mỗi công nghệ, cơ sở lý thuyết sẽ được giới thiệu kết hợp với các đoạn mã nguồn minh họa thực tế từ dự án.

2.1 Ngôn ngữ Kotlin và Jetpack Compose

2.1.1 Ngôn ngữ Kotlin

Kotlin là một ngôn ngữ lập trình kiểu tĩnh (statically typed) do JetBrains phát triển, hiện được Google công nhận là ngôn ngữ chính thức và ưu tiên hàng đầu (Kotlin-first) để phát triển ứng dụng Android. Kotlin giúp giảm thiểu mã rập khuôn (boilerplate code), tăng tính an toàn (đặc biệt là Null Safety) và hỗ trợ mạnh mẽ lập trình bất đồng bộ thông qua Coroutines.

2.1.2 Bộ công cụ Jetpack Compose

Jetpack Compose là bộ công cụ xây dựng giao diện người dùng (UI) hiện đại của Android theo mô hình khai báo (Declarative UI). Thay vì sử dụng XML để thiết kế giao diện như trước đây, lập trình viên sử dụng trực tiếp mã Kotlin để định nghĩa UI. Compose tự động cập nhật lại (recompose) giao diện khi trạng thái (state) thay đổi.



placeholder_compose.png

Hình 2.1: Mô hình hoạt động của Jetpack Compose

Đoạn mã sau trích xuất từ `DiscoverScreen.kt` minh họa cách định nghĩa một thành phần giao diện (Composable) để hiển thị danh sách truyện:

```
1 @Composable
2 fun DiscoverScreenContent(
3     modifier: Modifier = Modifier,
4     vm: DiscoverViewModel = hiltViewModel(),
5 ) {
6     val uiState by vm.uiState.collectAsStateWithLifecycle()
7
8     Scaffold(
9         topBar = { EditorialTopBar(...) },
10        containerColor = MaterialTheme.colorScheme.background,
11    ) { innerPadding ->
12        PullToRefreshBox(
13            isRefreshing = uiState.isLoading && uiState.items.isNotEmpty(),
```

```

14         onRefresh = { vm.loadDiscover() },
15         modifier = Modifier.padding(innerPadding)
16     ) {
17         // Hien thi danh sach truyen (LazyColumn / LazyVerticalGrid)
18     }
19 }
20 }

```

Hình 2.1: Định nghĩa giao diện màn hình Khám phá bằng Jetpack Compose

2.2 Kiến trúc Clean Architecture và mô hình MVVM

2.2.1 Clean Architecture

Clean Architecture là một mẫu kiến trúc phần mềm do Robert C. Martin (Uncle Bob) đề xuất, nhằm chia nhỏ hệ thống thành các tầng (layer) độc lập với nhau, lấy các quy tắc nghiệp vụ (Domain/Business logic) làm trung tâm. Trong dự án, ứng dụng được chia thành 3 tầng chính:

- **Domain Layer:** Chứa các định nghĩa Use Case, Model. Không phụ thuộc vào Android framework.
- **Data Layer:** Chứa Repository và Data Source (API, Database).
- **Presentation (UI) Layer:** Chứa giao diện (Compose) và ViewModel.



placeholder_clean_architecture.png

Hình 2.2: Sơ đồ kiến trúc Clean Architecture kết hợp MVVM

2.2.2 Mô hình MVVM (Model - View - ViewModel)

Mô hình MVVM được sử dụng ở tầng Presentation. **ViewModel** đóng vai trò giữ và quản lý dữ liệu cho **View** (UI), đồng thời xử lý các tương tác của người dùng để gọi xuống tầng **Domain/Data**. Trong Kotlin, ViewModel sử dụng **StateFlow** để phát (emit) dữ liệu lên View một cách phản ứng (reactive).

```
1 @HiltViewModel
2 class MangaDetailViewModel @Inject constructor(
3     private val mangaRepository: MangaRepository,
4     private val libraryRepository: LibraryRepository,
5 ) : ViewModel() {
6
7     private val _uiState = MutableStateFlow(MangaDetailUiState())
8     val uiState: StateFlow<MangaDetailUiState> = _uiState.asStateFlow()
9 }
```

```

10     private fun loadMangaDetail(mangaId: String) {
11         viewModelScope.launch(ioDispatcher) {
12             mangaRepository.getMangaDetail(mangaId).onSuccess { manga ->
13                 _uiState.update { it.copy(mangaDetail = manga) }
14             }
15         }
16     }
17 }

```

Hình 2.2: Khởi tạo ViewModel và quản lý StateFlow trong MangaDetailViewModel.kt

2.3 Dependency Injection với Hilt

Hilt là một thư viện tiêm phụ thuộc (Dependency Injection - DI) dành riêng cho Android, được xây dựng dựa trên nền tảng Dagger. Hilt giúp giảm thiểu các đoạn mã khởi tạo lặp đi lặp lại bằng cách tự động cung cấp các đối tượng (instance) cần thiết vào các class, ví dụ như tự động tiêm `Repository` vào `ViewModel`.



Hình 2.3: Cơ chế Tiêm phụ thuộc của Hilt trong Android

Bằng cách sử dụng các annotation như `@HiltViewModel` và `@Inject`, Hilt sẽ tự động khởi tạo và tiêm `MangaRepository` vào `MangaDetailViewModel` mà không cần lập trình viên phải gọi hàm khởi tạo (như đã thấy ở đoạn mã trên).

2.4 Cơ sở dữ liệu cục bộ với Room Database

Room là một thư viện ORM (Object-Relational Mapping) nằm trong hệ sinh thái Android Jetpack, đóng vai trò như một lớp trừu tượng bao bọc SQLite. Room giúp thao tác với cơ sở dữ liệu bằng cách sử dụng các đối tượng Java/Kotlin thay vì viết câu lệnh SQL thuần túy, đồng thời kiểm tra lỗi SQL ngay lúc biên dịch.

Trong ứng dụng, Room được sử dụng để lưu trữ danh sách truyện yêu thích và tiến trình đọc của người dùng. Các thao tác truy xuất dữ liệu được định nghĩa thông qua các Data Access Object (DAO).



Hình 2.4: Các thành phần cốt lõi của Room Database

```
1 @Dao
2 interface LibraryDao {
3     @Upsert
4     suspend fun upsert(item: LibraryItemEntity)
5
6     @Query("SELECT * FROM library_items WHERE sync_status != 'PENDING_DELETE'")
7     fun observeAll(): Flow<List<LibraryItemEntity>>
8
9     @Query("UPDATE library_items SET is_favorite = :isFavorite WHERE manga_id = :mangaId")
10    suspend fun updateFavorite(mangaId: String, isFavorite: Boolean): Int
11 }
```

Hình 2.3: Định nghĩa LibraryDao.kt sử dụng Room

2.5 Giao tiếp mạng với Retrofit và Coroutines

Retrofit là một Client HTTP an toàn về kiểu (type-safe) phổ biến nhất cho Android. Retrofit cho phép định nghĩa các REST API thông qua các Interface và Annotation. Trong dự án này, Retrofit được kết hợp với Kotlin Coroutines (`suspend fun`) và Kotlinx Serialization để gọi API MangaDex một cách bất đồng bộ và tự động chuyển đổi chuỗi JSON trả về thành các đối tượng Kotlin.



Hình 2.5: Sơ đồ luồng giao tiếp HTTP qua Retrofit

```
1 interface MangaDexApi {
2     @GET("manga")
3     suspend fun getMangaList(
4         @Query("limit") limit: Int,
5         @Query("offset") offset: Int,
6         @Query("includes[]") includes: List<String> = listOf("cover_art"),
7     ): MangaListResponseDto
8
9     @GET("manga/{mangaId}")
10    suspend fun getMangaDetail(
```

```
11     @Path("mangaId") mangaId: String,  
12     @Query("includes[]") includes: List<String> = listOf("cover_art"),  
13 ): MangaDetailResponseDto  
14 }
```

Hình 2.4: Sử dụng Retrofit định nghĩa các Endpoint trong MangaDexApi.kt

Ngoài ra, dự án sử dụng thư viện **Coil** (Coroutine Image Loader) được tích hợp sâu với Jetpack Compose thông qua hàm **AsyncImage** để tự động tải, giải mã và hiển thị hình ảnh (ảnh bìa truyện, trang truyện) trực tiếp từ các URL trả về của Retrofit.

Chương 3

Phương pháp đề xuất

Đối với các KLTN giải quyết bài toán nghiên cứu:

- Tiêu đề của chương thường là “Phương pháp đề xuất/Mô hình đề xuất cho bài toán XYZ”.
- Chương này cần trình bày được:
 - Tổng quan phương pháp/Kiến trúc tổng thể
 - Mô hình/thuật toán/phương pháp chi tiết
 - Cần lý giải được vì sao giải quyết bài toán như vậy, phân tích ưu, nhược điểm

Đối với các KTLN xây dựng hệ thống:

- Tiêu đề của chương thường là “Phân tích và thiết kế hệ thống XYZ”.
- Chương này cần trình bày được:
 - Phân tích yêu cầu
 - Kiến trúc tổng quan của hệ thống
 - Thiết kế hệ thống chi tiết: Biểu đồ ca sử dụng, các biểu đồ tuần tự, biểu đồ luồng hoạt động, biểu đồ lớp cho các ca sử dụng chính
 - Thiết kế CSDL

Lưu ý:

- **Không** đưa các bảng biểu/hình ảnh không liên quan vào KLTN
- Tiêu đề Bảng đặt trên bảng, tiêu đề Hình đặt dưới hình
- Toàn bộ hình ảnh, bảng biểu phải được tham chiếu và giải thích rõ ràng. **Không** chấp nhận các KLTN với các mục chỉ có hình/bảng không có giải thích/mô tả/thảo luận.

- **Không** viết KLTN theo dạng “gạch đầu dòng” để đảm bảo rõ ràng và liên mạch trong việc giải thích và lập luận.

Chương 4

Thực nghiệm và đánh giá

Đối với các KLTN giải quyết bài toán nghiên cứu:

- Chương này thường có tiêu đề “Thực nghiệm và đánh giá”
- Các nội dung chính cần trình bày:
 - Câu hỏi nghiên cứu
 - Dữ liệu và thiết lập thực nghiệm
 - Các phương pháp đối sánh
 - Độ đo đánh giá
 - Kết quả thực nghiệm
 - Phân tích và thảo luận kết quả

Đối với các KTLN xây dựng hệ thống:

- Chương này thường có tiêu đề “Cài đặt, kiểm thử và triển khai hệ thống”
 - Các nội dung chính cần trình bày:
 - * Môi trường triển khai và cài đặt
 - * Kịch bản kiểm thử
 - * Kết quả kiểm thử
 - * Phân tích và thảo luận kết quả

Lưu ý: Sinh viên cần mô tả phần mình cài đặt, phân tách với phần reuse

Kết luận

Tóm tắt bài toán, giải pháp và các kết quả đạt được